GenAI Developer Interview Prep — Gap-Area Q&A

(Reporting Analyst → GenAI Developer track)

This covers the topics identified as missing from your existing 25-question base: LangChain (Advanced), Vector Databases, Embeddings, RAG (Advanced), Prompt Engineering, LLM APIs, Production AI, Python, SQL, and Databricks.

---

1. LangChain (Advanced)

Q1. What is a Chain in LangChain? A Chain is a sequence of calls — to an LLM, a tool, a parser, or another chain — wired together so the output of one step becomes the input of the next. Chains let you compose simple building blocks (prompt → LLM → parser) into a reusable pipeline instead of writing that orchestration logic by hand each time.

Q2. What is LCEL (LangChain Expression Language) and why was it introduced? LCEL is a declarative syntax (using the | pipe operator) for composing LangChain components — prompt | llm | output_parser. It replaces the older class-based LLMChain/SequentialChain style. It was introduced because it gives you, for free: streaming support, async execution, batching, and parallel execution, without writing extra code — because every LCEL component implements the same Runnable interface.

Q3. What is a RunnableSequence? RunnableSequence is what gets created under the hood when you pipe Runnables together with |. It runs each component in order, passing the output of one as the input to the next. prompt | llm is syntactic sugar for RunnableSequence(first=prompt, last=llm).

Q4. What is an OutputParser and why do you need one? An OutputParser converts the raw string response from an LLM into a structured format your application can use — e.g., StrOutputParser (plain text), JsonOutputParser (dict), PydanticOutputParser (validated Python object). Without it, you're stuck parsing LLM text manually with regex or string splitting, which is brittle.

Q5. Difference between PromptTemplate and ChatPromptTemplate? PromptTemplate builds a single plain-text prompt string, typically used with completion-style (non-chat) models. ChatPromptTemplate builds a list of role-tagged messages (system, human, ai) for chat models like GPT-4 or Claude, which is what almost all modern LLM APIs expect.

Q6. What is Memory in LangChain, and what types exist? Memory lets a chain/agent retain context across turns in a conversation, since LLM calls are stateless by default. Common types:

- ConversationBufferMemory — stores the full raw conversation history
- ConversationBufferWindowMemory — keeps only the last *k* turns
- ConversationSummaryMemory — periodically summarizes older turns to save tokens
- ConversationSummaryBufferMemory — hybrid: recent turns verbatim + summary of older ones

Q7. What are Tools in LangChain? Tools are functions (search, calculator, SQL query, API call, retriever) that an LLM can decide to invoke to accomplish a task it can't do from parameters alone.

Each tool has a name, description, and input schema — the description is what the LLM reads to decide *when* to use it, so writing it clearly matters a lot.

Q8. Agents vs Chains — what's the difference? A Chain follows a fixed, predetermined sequence of steps every time. An Agent uses the LLM itself to *decide*, at runtime, which tool to call next and in what order, based on the intermediate results it's seeing (this is the "reasoning loop," e.g., ReAct). Chains are deterministic and predictable; Agents are dynamic and better suited to open-ended, multi-step tasks — but harder to debug and more expensive/slower.

Q9. When would you use a Chain instead of an Agent in production? When the workflow is well-defined and repeatable (e.g., "summarize this document, then extract 5 keywords, then classify sentiment"). Chains are cheaper, faster, more predictable, and easier to test — you should default to a Chain unless the task genuinely requires the LLM to make branching decisions about which tool to use next.

Q10. What's a common failure mode with LangChain Agents in production, and how do you mitigate it? Agents can loop indefinitely, call the wrong tool, or hallucinate tool inputs. Mitigations: set max_iterations / timeouts, constrain the toolset to only what's needed, use structured tool-calling (native function calling) instead of free-text ReAct parsing, and add a fallback/guardrail step that catches malformed tool calls.

---

2. Vector Databases

Q1. What is a vector database, in plain terms? A database optimized for storing high-dimensional embedding vectors and retrieving the most *similar* vectors to a query vector — instead of exact-match lookups like a relational DB, it does approximate nearest-neighbor (ANN) search.

Q2. FAISS — what is it and where does it fit? FAISS (Facebook AI Similarity Search) is a library, not a full database — it's an in-memory ANN indexing/search engine. It's extremely fast and free, but has no built-in persistence, metadata filtering, multi-tenancy, or network API out of the box — you build those yourself.

Q3. ChromaDB — what is it and where does it fit? Chroma is an open-source, lightweight vector database designed for simplicity and quick local/prototype use. It has built-in persistence, metadata filtering, and a simple Python API, making it popular for prototyping and small-to-medium RAG apps.

Q4. Pinecone — what is it and where does it fit? Pinecone is a fully managed, cloud-hosted vector database. You don't manage infrastructure — it handles scaling, replication, and uptime. It's a common choice for production apps that need to scale without a dedicated infra team, at the cost of a subscription and vendor lock-in.

Q5. Weaviate — what is it and where does it fit? Weaviate is an open-source vector database with built-in hybrid search (vector + keyword/BM25), GraphQL API, and modules for automatic vectorization. It can run self-hosted or as a managed cloud service, and is popular when you want hybrid search without stitching two systems together.

Q6. Milvus — what is it and where does it fit? Milvus is an open-source vector database built for very large-scale deployments (billions of vectors), with strong support for distributed architecture and multiple index types (IVF, HNSW, etc.). It's the go-to when scale is the primary constraint.

Q7. "Why would you choose FAISS over Chroma?" — how do you answer this? FAISS when: you need maximum raw search speed, full control over indexing algorithms, an in-memory/embedded solution with no server overhead, and you're willing to build persistence/filtering/APIs yourself. Chroma when: you want metadata filtering, persistence, and a simple developer experience out of the box, and you're prototyping or running a small-to-mid scale app without dedicated infra effort. In short: FAISS = raw performance + DIY; Chroma = convenience + batteries-included.

Q8. What indexing algorithms do vector databases typically use, and why? Common ones: Flat (brute-force, exact but slow at scale), IVF (Inverted File Index — clusters vectors, searches nearest clusters first), HNSW (Hierarchical Navigable Small World — graph-based, very fast approximate search, most widely used in production). The tradeoff is always speed/memory vs. recall accuracy.

Q9. What is metadata filtering in a vector DB and why does it matter? It's the ability to combine vector similarity search with structured filters (e.g., "only search documents where date > 2024 and department = 'finance'"). It matters because pure semantic search alone often surfaces relevant-but-wrong-context results — filtering narrows the search space before or after the similarity search.

Q10. How do you decide between a managed (Pinecone) vs self-hosted (Milvus/Weaviate/Chroma) vector DB for a production system? Consider: team's infra capacity (managed = less ops burden), data sensitivity/compliance (self-hosted = full data control), expected scale (Milvus for very large scale), cost model (subscription vs. compute+ops cost), and how much you need hybrid search or advanced features out of the box.

---

3. Embeddings

Q1. What are embeddings? Embeddings are dense numerical vector representations of text (or images/audio) that capture semantic meaning — texts with similar meaning end up close together in vector space, even if they don't share exact words.

Q2. How are embeddings generated? A neural network (often a transformer-based encoder like text-embedding-3-small, BGE, or Sentence-BERT) is trained so that semantically similar inputs produce vectors close together and dissimilar inputs produce vectors far apart. At inference, you pass text through this model once and get back a fixed-length vector (e.g., 1536 dimensions).

Q3. What is cosine similarity, and why is it commonly used for embeddings? Cosine similarity measures the angle between two vectors, ignoring their magnitude — it ranges from -1 (opposite) to 1 (identical direction). It's preferred for embeddings because what matters is the *direction* (semantic meaning) of the vector, not its length, which can vary based on text length or encoding artifacts.

Q4. Cosine similarity vs Euclidean distance — when would you use each? Cosine similarity is generally preferred for text embeddings because it's magnitude-invariant. Euclidean distance measures straight-line distance and is sensitive to vector magnitude — it's more common in domains like image embeddings or when the embedding model was specifically trained/normalized for it. Many embedding models normalize vectors to unit length, in which case cosine similarity and Euclidean distance produce the same ranking.

Q5. What are chunking strategies, and why does chunk size matter? Chunking splits large documents into smaller pieces before embedding, because embedding models have context limits and retrieval works best on focused, coherent passages. Common strategies:

- Fixed-size chunking (e.g., 500 tokens with overlap)
- Recursive character/text splitting (splits on paragraph → sentence → word as needed)
- Semantic chunking (splits at topic-boundary shifts detected via embeddings)
- Document-structure-aware chunking (split by headings/sections) Too small → loses context; too large → dilutes relevance and wastes tokens at retrieval time.

Q6. Why do we usually add overlap between chunks? To avoid cutting a sentence or idea in half at a chunk boundary, which would make both resulting chunks less coherent and less retrievable. A typical overlap is 10–20% of chunk size.

Q7. What is metadata filtering (in the context of embeddings/retrieval) and how does it improve retrieval quality? Attaching structured metadata (source, date, author, section, doc type) to each embedded chunk lets you constrain the search space before doing similarity search — e.g., "only search HR policy docs from 2024." This reduces irrelevant matches that are semantically similar but contextually wrong.

Q8. How do you choose an embedding model? Consider: dimensionality (higher = more expressive but more storage/compute), domain fit (general-purpose vs. domain-fine-tuned, e.g., legal/medical embeddings), context window (how much text it can embed in one call), cost/latency, and whether it needs to run locally (open-source like BGE/E5) vs. via API (OpenAI, Cohere).

Q9. What's the difference between sparse and dense embeddings? Dense embeddings (from neural models) are compact, low-dimensional, semantically rich vectors. Sparse embeddings (like TF-IDF or BM25 term weights) are high-dimensional, mostly-zero vectors based on exact word matches. Dense captures meaning; sparse captures exact keyword relevance — this distinction underlies hybrid search.

Q10. Can embeddings become "stale," and what do you do about it? Yes — if you update the embedding model, old vectors stored in your vector DB are no longer comparable to newly generated ones (different vector space), so you must re-embed your entire corpus after a model change. Teams typically version their embedding model and track which vectors were generated with which version.

---

4. RAG (Advanced)

Q1. Naive RAG — what is it and what are its limitations? Naive RAG is the basic pattern: embed the query → retrieve top-k similar chunks → stuff them into the prompt → generate an answer. Limitations: no query understanding/rewriting, no relevance verification, fixed retrieval count regardless of query complexity, and no correction if retrieval fails.

Q2. What is Advanced RAG? Advanced RAG adds optimization at each stage of the pipeline: pre-retrieval (query rewriting/expansion), retrieval (hybrid search, reranking), and post-retrieval (compression, filtering irrelevant chunks) — aimed at fixing naive RAG's weak points without changing the overall architecture.

Q3. What is Agentic RAG? Agentic RAG gives the LLM agency to decide *whether* to retrieve, *what* to retrieve, and *how many times* — it might issue multiple searches, use different tools/retrievers, or decide the retrieved info is insufficient and search again, rather than doing one fixed retrieval pass.

Q4. What is Corrective RAG (CRAG)? CRAG adds a relevance-evaluation step after retrieval: a lightweight evaluator grades whether the retrieved documents are actually relevant/sufficient. If they're not, it triggers a fallback — e.g., a web search, a query rewrite, or a different retrieval strategy — before generation, instead of blindly passing possibly-irrelevant context to the LLM.

Q5. What is GraphRAG? GraphRAG builds a knowledge graph from the source documents (entities and their relationships) and retrieves by traversing that graph, rather than relying purely on vector similarity. It's particularly strong for questions requiring multi-hop reasoning or understanding relationships between entities that pure semantic similarity search misses.

Q6. What is Hybrid Search? Hybrid search combines dense vector similarity search with sparse keyword search (like BM25), then merges the results (often via reciprocal rank fusion). It captures both semantic meaning *and* exact keyword/entity matches — useful because pure vector search sometimes misses exact terms like product codes, names, or acronyms that keyword search would catch.

Q7. What is Reranking, and why add it after retrieval? Reranking uses a second, typically more powerful (but slower) model — often a cross-encoder — to re-score and reorder the initial set of retrieved candidates for relevance to the query. Initial retrieval (bi-encoder/vector search) is fast but approximate; reranking trades a bit of latency for meaningfully better precision on the top results actually sent to the LLM.

Q8. What is Parent-Child (or Parent-Document) Retrieval? You embed and index small chunks (children) for precise similarity matching, but when a child chunk is retrieved, you return its larger parent chunk/document to the LLM for full context. This balances retrieval precision (small chunks match queries better) with generation quality (larger chunks preserve context).

Q9. What is a Multi-Query Retriever? Instead of using the user's single query as-is, an LLM generates several reformulated versions of the query (different phrasings/angles), each is used to retrieve independently, and the results are merged/deduplicated. This improves recall for queries where the user's exact wording doesn't closely match how the answer is phrased in the source documents.

Q10. What is a Self-Query Retriever? An LLM parses the user's natural-language query and automatically extracts both a semantic search component *and* structured metadata filters from it — e.g., "reviews from 2023 that mention battery life" becomes a vector search for "battery life" plus a metadata filter year=2023, without the user having to specify the filter explicitly.

Q11. How would you decide between plain vector search, hybrid search, and GraphRAG for a given use case? Plain vector search for general semantic Q&A over unstructured text. Hybrid search when queries contain exact terms, IDs, codes, or names that must match precisely. GraphRAG when the domain has rich entity relationships and questions require multi-hop reasoning (e.g., "who reports to the person who approved this contract").

---

5. Prompt Engineering (Techniques)

Q1. Zero-shot prompting — what is it? Asking the model to perform a task with no examples given, relying purely on its pretrained knowledge and the instruction itself (e.g., "Classify this review as positive or negative: ...").

Q2. One-shot prompting — what is it? Providing exactly one example of the input/output pattern before asking the model to perform the task on a new input, to anchor the expected format and style.

Q3. Few-shot prompting — what is it, and when does it help most? Providing several (typically 2–8) input/output examples in the prompt to demonstrate the desired pattern. It helps most when the output format is unusual/specific, the task is ambiguous from instructions alone, or you need consistent style/structure — at the cost of extra tokens.

Q4. Chain of Thought (CoT) prompting — what is it? Instructing the model to reason step-by-step before giving a final answer (e.g., "Let's think step by step"), rather than jumping straight to the answer. This measurably improves performance on tasks requiring multi-step reasoning, like math or logic problems, because it gives the model "room" to work through intermediate steps.

Q5. ReAct prompting — what is it? ReAct (Reason + Act) interleaves reasoning traces with actions (tool calls) — the model reasons about what it needs, takes an action (e.g., a search), observes the result, and reasons again, in a loop, until it reaches a final answer. This is the pattern underlying most tool-using agents.

Q6. Tree of Thought — what is it and how does it differ from CoT? Where CoT follows a single linear reasoning path, Tree of Thought has the model explore multiple different reasoning branches in parallel, evaluate which branches look most promising, and backtrack from dead ends — closer to how a person might explore several approaches to a hard problem before committing to one.

Q7. Role prompting — what is it and why does it work? Assigning the model a persona or role (e.g., "You are a senior SQL performance engineer") to steer tone, vocabulary, and the type of reasoning it applies. It works because it conditions the model toward a narrower, more relevant slice of its training distribution.

Q8. XML prompting — what is it and why is it useful? Structuring prompt content using XML-style tags (e.g., <context>...</context>, <question>...</question>, <instructions>...</instructions>) to clearly delineate different parts of a prompt. It's especially effective with Claude models, and it reduces ambiguity about where instructions end and content begins, which lowers the chance of the model conflating data with directives.

Q9. How do you choose between zero-shot, few-shot, and CoT for a given task? Start with zero-shot for simple, well-understood tasks (cheapest, fastest). Move to few-shot when output format/style needs to be tightly controlled or the task is non-obvious. Add CoT when the task requires multi-step reasoning (math, logic, multi-constraint decisions) — and these can be combined (few-shot + CoT together is common and effective).

Q10. What's a common mistake in prompt engineering interviews people make when explaining these techniques? Treating them as interchangeable "tricks" instead of explaining *why* each works and *when* to use it — a strong answer always ties the technique to the specific failure mode it addresses (e.g., "CoT helps because it gives the model intermediate computation steps instead of forcing an immediate answer").

---

6. LLM APIs

Q1. How do you call the OpenAI API for a chat completion, at a high level? Send a POST request to the chat completions endpoint with a model name, a list of role-tagged messages (system/user/assistant), and parameters like temperature and max_tokens; the response returns generated message(s) plus token usage metadata.

Q2. How does Groq's API differ from OpenAI's, conceptually? Groq is largely OpenAI-API-compatible in interface, but its differentiator is inference speed — it runs open-source models (Llama, Mixtral, etc.) on custom LPU hardware optimized for extremely low-latency token generation, rather than offering its own proprietary model family.

Q3. What is streaming in the context of LLM APIs, and why use it? Instead of waiting for the full response to be generated before returning anything, the API sends tokens back incrementally as they're generated. It's used to reduce perceived latency in user-facing apps (the user sees text appearing immediately) and to allow early-exit logic (e.g., stop generation if a stop condition is met mid-stream).

Q4. What is token usage, and why do developers track it closely? Every API call consumes input tokens (prompt) and output tokens (completion), and pricing is per-token. Tracking usage matters for cost control, for staying under context-window limits, and for diagnosing runaway costs from things like unbounded conversation history or agent loops.

Q5. What are common cost optimization strategies for LLM apps? Use a smaller/cheaper model where quality allows (route only hard queries to the best model); cache repeated or semantically similar responses; trim conversation history/summarize older turns; reduce retrieved context to only what's relevant (reranking helps here); batch requests where the API supports it; and set sensible max_tokens caps.

Q6. What is a retry mechanism, and why does it matter for production LLM apps? Logic that automatically retries a failed API call (network errors, timeouts, transient 5xx errors) — usually with exponential backoff — instead of failing the whole request outright. It matters because LLM APIs occasionally have transient failures under load, and naive single-attempt calls make your app unnecessarily fragile.

Q7. What is rate limiting, and how do you handle it in your application? LLM providers cap requests-per-minute and/or tokens-per-minute per account/API key. You handle it by implementing client-side throttling/queuing, respecting 429 responses with backoff, and in some cases batching or spreading requests, so your app degrades gracefully instead of erroring out under load.

Q8. What's the difference between synchronous and streaming API error handling? With a normal request, an error either happens up front or you get a clean response. With streaming, an error can occur *mid-stream* after you've already shown the user partial output — so your app needs to handle partial responses gracefully (e.g., show what was generated plus a clear "generation interrupted" state) rather than assuming an error means nothing was returned.

Q9. How would you design retry logic so it doesn't make rate-limiting worse? Use exponential backoff with jitter (randomized delay) rather than fixed-interval retries, so multiple failed requests

don't all retry at the same instant and hammer the API simultaneously; also cap total retry attempts and surface a clear failure after that.

Q10. What's the difference between a completions API and a chat/messages API? Older-style completion APIs take a single raw text prompt and continue it. Modern chat/messages APIs take a structured list of role-tagged messages (system/user/assistant), which lets the model distinguish instructions from conversation history from user input — this structure is what most current-generation models (GPT-4, Claude) are trained around.

---

7. Production AI

Q1. Why is caching important in an LLM application, and what do you cache? LLM calls are slow and expensive, so caching avoids recomputation for repeated/similar requests. You can cache: exact-match query→response pairs, semantically similar queries (via embedding similarity threshold), or intermediate steps like retrieval results.

Q2. What role does Redis play in a GenAI production stack? Redis is commonly used as a fast in-memory cache layer (for LLM response caching, session/conversation state, or rate-limit counters) and sometimes as a lightweight vector store (via RediSearch) for smaller-scale similarity search — its main value is low-latency read/write.

Q3. What should you log in an LLM application, beyond standard app logs? Prompts sent, responses received, token counts, latency per call, model/version used, retrieval results (for RAG), user feedback (thumbs up/down), and any tool calls made — this is essential for debugging bad outputs after the fact and for building evaluation datasets.

Q4. What is "monitoring" in the LLM context, and how does it differ from traditional app monitoring? Beyond typical infra metrics (latency, error rate, uptime), LLM monitoring also tracks output *quality* signals over time — hallucination rate, user feedback trends, drift in response length/style, cost per request, and retrieval relevance — because an LLM app can be "up" (200 OK) while silently producing worse answers.

Q5. What is LLM evaluation, and what approaches exist? Systematically measuring output quality against a benchmark or rubric. Approaches: human evaluation (gold-standard but slow/expensive), LLM-as-judge (using a strong model to score outputs against criteria), reference-based metrics (BLEU/ROUGE/exact-match, mostly for narrow tasks), and task-specific automated checks (e.g., does the generated SQL actually execute and match expected results).

Q6. How do you detect hallucination in a RAG system? Techniques include: checking whether claims in the generated answer are actually supported by the retrieved context (via an LLM-as-judge "faithfulness" check, or NLI-style entailment models), flagging answers with no supporting citations, and comparing generated facts against a trusted source/ground truth where available.

Q7. What is prompt versioning, and why does it matter in production? Treating prompts like code — tracking changes over time (git or a prompt-management tool), tagging which version is in production, and being able to roll back. It matters because a "small" prompt tweak can silently change behavior/quality significantly, and without versioning you lose the ability to diagnose regressions or A/B test changes.

Q8. What are guardrails in an LLM application? Automated checks that constrain model input/output — e.g., blocking PII in inputs, filtering toxic/unsafe outputs, validating structured output against a schema, enforcing topic boundaries (won't answer off-topic questions), and catching prompt injection attempts. Libraries like Guardrails AI or NeMo Guardrails implement this pattern.

Q9. What is "observability" for LLM apps, and how is it different from monitoring? Monitoring tells you *that* something's wrong (a metric crossed a threshold); observability gives you the ability to *understand why* — tracing a specific request end-to-end through retrieval, prompt construction, model call, and post-processing, so you can debug a single bad output, not just see aggregate trends. Tools like LangSmith, Langfuse, or Arize Phoenix are built for this.

Q10. What does CI/CD look like for LLM applications, and how does it differ from traditional software CI/CD? It includes the usual (tests, linting, deployment pipelines) plus LLM-specific steps: running an evaluation suite against a golden dataset before deploying a prompt/model change, regression-testing output quality (not just "does it compile"), and often a staged rollout since output quality can be subjective and harder to gate with a single pass/fail test than traditional code.

Q11. How would you design a fallback strategy if your primary LLM API goes down or is rate-limited? Use a secondary provider/model as fallback (with a compatible interface), implement circuit-breaker logic to stop hammering a failing provider, degrade gracefully (e.g., serve cached/simpler responses), and alert on sustained failures rather than silently failing user requests.

---

8. Python Questions

Q1. What are decorators in Python, and give a practical GenAI-related example? A decorator wraps a function to add behavior without modifying its code, using @decorator_name syntax. Practical example: a @retry(max_attempts=3) decorator around an LLM API call function, or a @cache decorator to memoize repeated embedding calls.

Q2. What are generators, and why are they useful for LLM streaming? A generator is a function that uses yield to produce a sequence of values lazily, one at a time, without holding the whole sequence in memory. They're a natural fit for LLM response streaming — you yield each token/chunk as it arrives from the API rather than waiting for and buffering the full response.

Q3. What is async programming in Python, and why does it matter for GenAI apps? async/await lets you write non-blocking code — while waiting on an I/O-bound operation (like an LLM API call or a DB query), the program can do other work instead of sitting idle. It matters a lot in GenAI apps because LLM API calls are slow (seconds), and async lets you handle many concurrent requests (e.g., multiple users, or parallel retrieval + multiple tool calls) efficiently.

Q4. How do you handle exceptions when calling an external API like OpenAI's? Wrap the call in try/except, catch specific exception types the SDK raises (rate limit errors, timeout errors, authentication errors) separately so you can respond appropriately to each (retry vs. fail vs. alert), and avoid bare except: clauses that swallow errors silently.

Q5. Explain OOP concepts (encapsulation, inheritance, polymorphism) with a GenAI-relevant example. Encapsulation: a VectorStoreClient class hides connection details behind simple add()/query() methods. Inheritance: a BaseRetriever class defines a common interface, and

FAISSRetriever/ChromaRetriever subclasses implement it differently. Polymorphism: your RAG pipeline code calls retriever.retrieve(query) without caring which concrete retriever class it's actually using.

Q6. Multithreading vs multiprocessing — what's the difference, and which matters more for LLM app work? Multithreading runs multiple threads within one process sharing memory — good for I/O-bound work (waiting on network calls) but limited by Python's GIL for CPU-bound work. Multiprocessing runs separate processes with separate memory, bypassing the GIL — good for CPU-bound work (e.g., local embedding generation, heavy text preprocessing). For LLM app work, which is mostly I/O-bound (waiting on API responses), async or multithreading is typically more relevant than multiprocessing.

Q7. What is the GIL (Global Interpreter Lock), and why does it come up in these interviews? The GIL ensures only one thread executes Python bytecode at a time in a single process, which means CPU-bound multithreaded Python code doesn't get true parallelism. It comes up because interviewers want to check you understand why async/multiprocessing are often preferred over plain threading for certain workloads in Python specifically.

Q8. How would you implement a simple retry-with-backoff decorator for LLM API calls? Conceptually: the decorator catches specific transient exceptions, waits an exponentially increasing delay (optionally with random jitter) between attempts, retries up to a max count, and re-raises (or returns a fallback) if all attempts fail — this is commonly done with the tenacity library rather than hand-rolled in production code.

Q9. What's the difference between *args and **kwargs, and where might you see them in LangChain-style code? *args collects extra positional arguments into a tuple; **kwargs collects extra keyword arguments into a dict. You'll see this heavily in LangChain/framework code where wrapper functions need to pass arbitrary parameters through to an underlying API call without knowing all of them in advance.

Q10. Why might you use a context manager (with statement) in a GenAI data pipeline? To guarantee cleanup happens even if an error occurs — e.g., closing a DB connection, releasing a file handle after reading a document for chunking, or managing a temporary resource — without needing manual try/finally boilerplate every time.

---

9. SQL Questions

Q1. Explain the different types of JOINs. INNER JOIN returns only matching rows in both tables. LEFT JOIN returns all rows from the left table plus matches from the right (NULL where no match). RIGHT JOIN is the mirror of LEFT. FULL OUTER JOIN returns all rows from both, matched where possible. CROSS JOIN returns the Cartesian product of both tables.

Q2. What are window functions, and why are they useful? Window functions (e.g., ROW_NUMBER(), RANK(), SUM() OVER (...)) perform calculations across a set of rows related to the current row, *without* collapsing rows the way GROUP BY does. They're useful for running totals, rankings, and comparisons "within a partition" while still returning row-level detail.

Q3. What is a CTE (Common Table Expression), and why use one over a subquery? A CTE (WITH x AS (...)) is a named temporary result set defined before the main query. It improves readability for complex queries, allows reuse of the same result set multiple times in one query, and supports recursion (recursive CTEs) — which a plain subquery can't do as cleanly.

Q4. How would you rank rows within groups (e.g., top 3 products per category)? Use a window function like RANK() or ROW_NUMBER() OVER (PARTITION BY category ORDER BY sales DESC) inside a CTE or subquery, then filter WHERE rank <= 3 in the outer query (you can't filter directly on a window function in the same SELECT level as it's computed).

Q5. RANK() vs DENSE_RANK() vs ROW_NUMBER() — what's the difference? ROW_NUMBER() assigns strictly sequential unique numbers regardless of ties. RANK() gives tied rows the same rank but skips subsequent rank numbers (1,1,3). DENSE_RANK() gives tied rows the same rank without skipping (1,1,2).

Q6. What's the difference between WHERE and HAVING? WHERE filters individual rows *before* aggregation/grouping happens. HAVING filters *after* aggregation, on the results of aggregate functions like SUM() or COUNT() — you can't use an aggregate function in a WHERE clause.

Q7. What are common query optimization techniques? Add indexes on columns used in JOINs/WHERE/ORDER BY; avoid SELECT *; filter as early as possible; avoid functions on indexed columns in WHERE clauses (which prevents index usage); use EXPLAIN/query plans to find bottlenecks; denormalize selectively for read-heavy workloads; and avoid unnecessary subqueries when a JOIN would do.

Q8. How would you find duplicate rows in a table using SQL? Group by the columns that define a "duplicate" and filter with HAVING COUNT(*) > 1 — e.g., SELECT email, COUNT(*) FROM users GROUP BY email HAVING COUNT(*) > 1.

Q9. What is the difference between UNION and UNION ALL? UNION combines result sets and removes duplicates (which requires an implicit sort/dedup, so it's slower). UNION ALL combines result sets and keeps all rows including duplicates, which is faster since no deduplication step is needed.

Q10. Given a "text-to-SQL" GenAI use case, what SQL knowledge do interviewers specifically probe? Whether you understand *why* the generated SQL might be wrong (ambiguous column names across joined tables, incorrect aggregation grouping, missing date filters), how you'd validate LLM-generated SQL before executing it against production data (e.g., running it against a read replica, using EXPLAIN to sanity-check the plan first, restricting to SELECT-only), and how you'd handle schema context (providing table/column descriptions in the prompt so the LLM picks the right joins).

---

10. Databricks

Q1. What is Unity Catalog? Unity Catalog is Databricks' centralized governance layer for data and AI assets — it manages access control, auditing, lineage, and discovery across all workspaces in one place, using a three-level namespace (catalog.schema.table) instead of the older per-workspace metastore model.

Q2. What is Delta Lake? Delta Lake is an open-source storage layer that brings ACID transactions, schema enforcement, and versioning (time travel) to data stored in cloud object storage (like S3/ADLS), on top of the Parquet file format. It's what turns a plain data lake into a more reliable "lakehouse."

Q3. What is a Delta Table? A Delta Table is a table stored using the Delta Lake format — it supports ACID transactions, MERGE/UPDATE/DELETE operations (which plain Parquet doesn't support natively), schema evolution, and time travel (querying a previous version of the table).

Q4. What are Databricks Clusters? Clusters are the compute resources (a set of VMs) that run your notebooks, jobs, and SQL queries. They can be all-purpose (interactive development) or job clusters (spun up for a scheduled job and torn down after), and support autoscaling to add/remove worker nodes based on load.

Q5. What is a SQL Warehouse? A SQL Warehouse (formerly SQL Endpoint) is compute specifically optimized for running SQL queries (e.g., from BI tools or the Databricks SQL editor) rather than general-purpose Spark notebook workloads — it's tuned for low-latency, high-concurrency SQL rather than large distributed ETL jobs.

Q6. What are Notebook Jobs? A Notebook Job is a scheduled or triggered execution of a Databricks notebook (or a chain of notebooks/tasks) as a production pipeline — used to automate recurring workflows like nightly ETL, embedding generation, or model retraining, with monitoring, retries, and alerting built in.

Q7. What are Delta Live Tables (DLT)? DLT is a declarative framework for building ETL pipelines on Databricks — you define the *desired end state* of your tables (and data quality expectations) and DLT manages the underlying orchestration, incremental processing, and error handling, rather than you hand-writing imperative pipeline logic.

Q8. How might Databricks fit into a GenAI/RAG architecture specifically? Common pattern: raw documents land in a Delta table (bronze layer), get cleaned/chunked in a subsequent pipeline stage (silver), embeddings are generated (often via a Databricks-hosted or external embedding model) and stored either in a Delta table or synced to a vector index (Databricks Vector Search), and the whole ingestion pipeline is orchestrated as a scheduled Job or DLT pipeline — giving you governed, versioned, reproducible RAG data prep.

Q9. What is "time travel" in Delta Lake, and why is it useful? The ability to query a Delta table as it existed at a previous version or timestamp (SELECT * FROM table VERSION AS OF 5), because Delta Lake keeps a transaction log of all changes. Useful for debugging data issues, auditing, and reproducing a RAG pipeline's exact state at a point in time.

Q10. What's the difference between a workspace-level metastore and Unity Catalog governance? The legacy Hive metastore was scoped per-workspace, meaning permissions and catalogs didn't cleanly share across workspaces. Unity Catalog centralizes governance across *all* workspaces in an account, with fine-grained access control down to the row/column level, unified audit logs, and cross-workspace data lineage — which matters a lot once an org has more than one team/workspace using the same underlying data.

How to use this document

- Treat each section as a mini flash-card set — cover the answer and try to reconstruct it in your own words.
- For the "advanced" sections (RAG Advanced, LangChain Advanced), be ready to draw the pipeline on a whiteboard, not just recite definitions.
- For Python/SQL/Databricks, practice writing the actual code/query, not just explaining it conceptually — these are often asked as live coding, not verbal Q&A.